

|                  |                                    |
|------------------|------------------------------------|
| <b>Doc. no.:</b> | P2136R0                            |
| <b>Date:</b>     | 2020-3-2                           |
| <b>Audience:</b> | LEWG                               |
| <b>Reply-to:</b> | Zhihao Yuan <zy at miator dot net> |

# invoke<R>

## Introduction

This paper proposes `invoke<R>`, a variant of `std::invoke` that allows specifying the return type, realizing the semantics of `INVOKE<R>` rather than `INVOKE`.

## History

When `std::invoke` was introduced in N4169<sup>[1]</sup>, `invoke<R>` was dropped from the proposal with the belief that such a form is unnecessary, stating:

Mentioned version is leftover from TR1 implementation, where the result type was determined using `result_of` protocol or has to be specified at the call side and, after the introduction of type interference in C++11, it becomes obsolete.

But in 2015, LWG 2420<sup>[2]</sup> was applied to the working draft. `INVOKE(f, args..., void)`'s capability of casting away the return value is confirmed and specified.

In 2016, the author of this paper opened LWG 2690<sup>[3]</sup>, proposing `std::invoke<R>`. The author of N4169 commented on that issue, confirmed that:

The lack of `invoke<R>` was basically a result of the concurrent publication of the never revision of the paper and additional special semantics of `INVOKE(f, args..., void)`.

In 2017, `INVOKE(f, args..., void)` gained the current spelling `INVOKE<R>(f, args...)` in P0604R0<sup>[4]</sup>. In the same paper, all the new invocation traits get `_r` variants that allow specifying the return type.

In 2018, `std::visit<R>`<sup>[5]</sup> is added to the working draft. The usefulness of `INVOKE<R>` keeps getting attention.

# Discussion

---

## How useful `invoke<R>` is?

`invoke<R>(...)` does three things that `std::invoke(...)` does not:

1. In a call forwarder that allows specifying the return type or the full signature, putting `void` as the return type naturally discards the return value, as implied by `std::is_invocable_r` and `std::is_nothrow_invocable_r`.
2. When `R` is not `cv void`, you can specify a compatible return type that is different from the callable entity. For example, you can request a function that returns `T&&` to return a prvalue of type `T` by using `invoke<T>`.
3. If the callable entity has overloaded call operators that may return different types, they may agree on a return type that allows you to specify.

## Can we avoid ambiguity when `R` is the callable type?

1. We can name it `invoke_r` if we want to, although neither `std::bind` nor `std::visit` does this.
2. We may also prevent the existing `std::invoke` from deducing its first template parameter by prepending a `int = 0` (deduction guard (<https://github.com/eggs-cpp/variant/blob/master/include/eggs/variant/variant.hpp#L2384>)) if that does not raise ABI concerns.
3. Constraining the new signature is doable but tricky. The wording needs to match value categories.

## Wording

---

The wording is relative to N4849.

Modify 20.14.1 [functional.syn], header `<functional>` synopsis, as indicated:

```
namespace std {  
    // 20.14.4 [func.invoke], invoke  
    template<class F, class... Args>  
        constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)  
            noexcept(is_nothrow_invocable_v<F, Args...>);  
    template <class R, class F, class... Args>  
    constexpr R invoke(F&& f, Args&&... args)  
    noexcept(is_nothrow_invocable_r_v<R, F, Args...>);
```

Add the following sequence of paragraphs after 20.14.4 [func.invoke]/1 as indicated:

```
template <class R, class F, class... Args>
constexpr R invoke(F&& f, Args&&... args)
noexcept(is_nothrow_invocable_r_v<R, F, Args...>);
```

Constraints: `is_invocable_r_v<R, F, Args...> .`

Returns: `INVOKE<R>(std::forward<F>(f), std::forward<Args>(args)...) (20.14.3 [func.require]).`

## References

1. Kamiński, Tomasz. *A proposal to add invoke function template (Revision 1)*. <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4169.html> (<http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2014/n4169.html>) ↩
2. Bergé, Agustín. LWG 2420 *function<void(ArgTypes...)> does not discard the return value of the target object*. <https://cplusplus.github.io/LWG/issue2420> (<https://cplusplus.github.io/LWG/issue2420>) ↩
3. Yuan, Zhihao. LWG 2690 *invoke<R> .* <https://cplusplus.github.io/LWG/lwg-active.html#2690> (<https://cplusplus.github.io/LWG/lwg-active.html#2690>) ↩
4. Krügler, Daniel et al. *Resolving GB 55, US 84, US 85, US 86*. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0604r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0604r0.html>) ↩
5. Park, Michael and Agustín Bergé. *visit<R> : Explicit Return Type for visit .* <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0655r1.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0655r1.pdf>) ↩